

Filter-Aware Fine-Tuning for Safe Humanoid Whole-Body Tracking

Pranit Mohnot¹, Christian Helten², Daniele Gammelli^{1,3}, Marco Pavone^{1,4}

Abstract—Safe whole-body motion is essential for deploying humanoid robots in unstructured environments. Modern humanoid control commonly separates reference specification from execution, with a planner, teleoperator, or motion generator providing a reference that a reinforcement-learning policy tracks through dynamically feasible whole-body control. Runtime safety filters, such as control barrier functions (CBFs), offer a promising approach for enforcing newly introduced constraints via interventions on the tracker’s outputs. We show, however, that treating the tracking policy and safety filter independently induces fundamental mismatches, as filtering alters both the executed actions and the induced state distribution. We study this policy–filter interface through case studies that isolate dynamics, objective, and information mismatches, highlight their root causes, and use these insights to develop *CoFiT* (Constrained Filter-aware Tuning), a filter-aware fine-tuning method for pretrained trackers. Across diverse constraint scenes, CoFiT reduces violation time relative to filter-only training by 91% on TWIST2 and 21% on SONIC, while requiring smaller safety filter corrections. On Unitree G1 hardware, CoFiT reduces violation time by 83% for TWIST2 and completes every trial without operator intervention, whereas 50% of baseline trials require an operator stop. Together, these results provide actionable insights into policy–filter interactions and establish design principles for integrating learned trackers with runtime safety filters.

I. INTRODUCTION

Humanoid robots are rapidly moving toward deployment in homes, warehouses, hospitals, and other human-centered environments, where safe operation requires coordinating the full body under task- and environment-dependent constraints. Whole-body control increasingly relies on learning-based motion tracking, in which a motion source, such as a teleoperator, recorded demonstration, or kinematic motion generator, provides a reference trajectory that a reinforcement-learning (RL) tracking policy translates into joint commands consistent with the robot’s dynamics [1], [2], [3]. While this control architecture enables expressive whole-body motion, ensuring its safety in real-world environments remains challenging. A reference trajectory specifies the intended motion but does not, in general, encode every constraint of the environment in which that motion will be executed. Obstacles, workspace limits, and other safety requirements may be absent from the motion source, emerge only at runtime, or change after the tracking policy has been trained.

Two broad strategies have emerged for enforcing safety in whole-body control. The first internalizes safety within the learned tracking policy, for example by training with penalties on constraint violations or barrier-based rewards that encourage avoidance of unsafe states [4]. This approach tightly couples safety to training, as accommodating a new

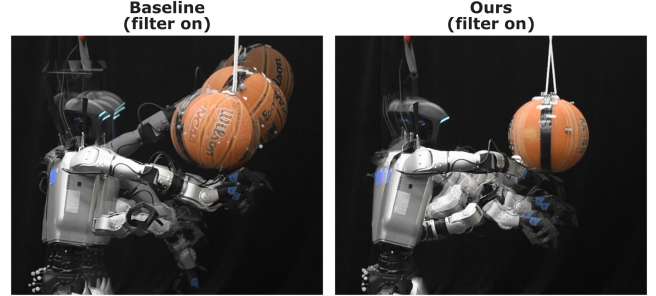


FIG. 1: Whole-body tracking with an active safety filter (avoid the ball). The continued-training baseline (left) and CoFiT (right) track the same reaching reference. Five of ten baseline trials required an operator stop; CoFiT completed all ten. Overlaid poses are successive stills from one trial.

constraint generally requires retraining, and the resulting policy does not by itself provide a runtime guarantee. The second strategy places a safety filter downstream of the tracking policy to correct its actions at runtime [5], [6]. This modular approach leaves the nominal policy unchanged and can enforce constraints that were not present during training.

Control barrier functions (CBFs) are widely used to construct such safety filters. A CBF expresses a constraint as a condition on the control input, and the filter modifies the tracker’s proposed command only as much as needed to satisfy that condition [5]. Whole-body formulations can additionally account for contact constraints and task priorities when selecting the correction [7]. This modularity is appealing, as the tracker can be treated as a fixed nominal controller, while the filter supplies policy-agnostic corrections, seemingly allowing the two components to be designed independently.

In this work, we show that treating the tracking policy and safety filter independently induces fundamental mismatches in closed-loop execution (Fig. 1). The safety filter changes the tracker’s outputs even though the tracker was trained under an unfiltered closed loop. As a result, both the actions executed by the robot and the states induced by those actions differ from those encountered during training. Through systematic case studies, we identify three key sources of incompatibility at the policy–filter interface: *dynamics mismatch*, *objective mismatch*, and *information mismatch*. Guided by these findings and their underlying causes, we develop *CoFiT* (Constrained Filter-aware Tuning), a method that adapts a pretrained tracker to an existing CBF filter. CoFiT fine-tunes the tracker with filters active and supplies it with compact constraint information, temporal feedback, and constraint-aware penalties. This allows the tracker to anticipate filter interventions and respond with coordinated, constraint-compatible motion while retaining its original tracking objective where possible.

Our contributions are as follows:

¹Stanford University, ²Technical University of Munich, ³Italian Institute of Artificial Intelligence (IAI), ⁴NVIDIA

- We provide a systematic analysis of the policy–filter interface, identifying dynamics, objective, and information mismatches between pretrained tracking policies and runtime CBF filters and relating them to specific training and interface design choices.
- We introduce *CoFiT*, a filter-aware fine-tuning method that adapts pretrained whole-body tracking policies to an existing CBF filter using action-space constraint observations, temporal feedback, and constraint-aware rewards.
- We validate *CoFiT* through extensive ablations and experiments across multiple base trackers in simulation and on hardware, demonstrating fewer constraint violations, smaller filter corrections, and smoother motion.

II. RELATED WORK

We review prior work along two complementary directions: learning in the presence of runtime safety filters, and incorporating safety constraints across the control hierarchy for legged and humanoid robots.

A. Learning with safety filters

Safety filters separate nominal task execution from explicit safety enforcement by admitting or modifying a policy’s proposed actions according to safety constraints. CBF-based methods enforce local model-based inequality constraints on proposed controls, while predictive safety filters evaluate candidate inputs through backup trajectories and terminal safe sets [8], [6]. Several works train policies directly on the resulting filtered system so that they can adapt to these interventions. CBF-RL combines training-time filtering with rewards based on barrier conditions and filter corrections [9], while Safety Filtering While Training learns from filtered transitions and penalizes corrective interventions with a predictive filter retained during evaluation [10]. Related approaches differentiate through CBF layers [11] or expose barrier and Lie-derivative quantities to the policy when selecting CBF-related parameters [12]. These methods show that policies can benefit from being trained with knowledge of the safety mechanism, but they do not address how a pretrained whole-body tracker should be adapted to an already deployed runtime filter, nor which constraint representations and feedback signals are most useful for doing so. Our work targets this setting by conditioning the nominal whole-body tracker on a compact action-space representation of constraint information and systematically studying how this constraint information, along with temporal feedback and training objectives, affects behavior under the inherited filter.

B. Constraints across the control hierarchy

Safety constraints can be incorporated at multiple levels of a legged or humanoid control stack, including motion planning, reference generation, and whole-body command execution. Planning and tracking layers can share CBF constraints, while filters above learned locomotion controllers can modify high-level references [13], [14]. At the command level, humanoid controllers combine barrier constraints with full dynamics and contact variables [15], [16]. Task-consistent formulations further account for how safety corrections

affect the primary motion and posture, as in OSCBF for manipulation and ConstrainedMimic for pretrained humanoid tracking [7], [17]. Other works study the role of safety-related perception. For example, PAC-MAN combines depth with barrier-based rewards for reactive humanoid dodgeball [4]. Collectively, these works establish effective mechanisms for injecting safety constraints at different levels of the control hierarchy, but largely treat the nominal tracking policy and downstream whole-body filter as separately designed components. Our work focuses on this interface: we retain the existing whole-body safety filter and study how the tracking policy that supplies its nominal commands should be adapted to account for the filter’s constraints and interventions.

III. PRELIMINARIES

In this section, we first briefly review the relative-degree-two CBF formulation used to encode safety constraints (Section III-A), and then describe the whole-body filtering framework through which these constraints modify the tracking policy’s nominal commands (Section III-B).

A. Relative Degree Two Control Barrier Functions

A control barrier function expresses a state constraint as a condition on the control input [5]. Let $h_i(z) \geq 0$ define an admissible region for a control-affine model $\dot{z} = f(z) + g(z)u$, with Lie derivatives $L_f h_i = \nabla h_i^\top f$ and $L_g h_i = \nabla h_i^\top g$ describing drift and input sensitivity. Position constraints under torque control have $L_g h_i = 0$, since the input enters through acceleration. Such constraints have relative degree two and can be enforced through:

$$\begin{aligned} h_{2,i} &= L_f h_i + \alpha(h_i), \\ L_f h_{2,i} + L_g h_{2,i} u + \alpha_2(h_{2,i}) &\geq 0, \end{aligned} \quad (1)$$

where α, α_2 are extended class- $\mathcal{K}_\infty$ functions [7], [18].¹ The intermediate barrier $h_{2,i}$ combines the barrier value with approach rate, allowing the CBF condition to regulate not only proximity to the constraint boundary but also how rapidly the system approaches it.

B. Whole-Body Constrained Tracking

A whole-body filter should choose a constraint correction without unnecessarily disturbing contact support or the motion task. Operational-space formulations therefore measure intervention according to its effects on the task rather than through a uniform distance in torque space [17]. ConstrainedMimic extends this principle to contact-consistent humanoid dynamics by weighting changes in contact forces, task accelerations, and posture accelerations [7]. We use this framework to construct all safety filters in this work.

For a spherical body proxy i and a static spherical obstacle j , we define the distance barrier:

$$h_{ij}(q) = \|p_i(q) - c_j\|_2 - r_i - r_j - d_{\text{safe}}, \quad (2)$$

where $p_i(q)$ and c_j denote the sphere centers, r_i and r_j their radii, and $d_{\text{safe}} \geq 0$ the desired safety buffer. For other obstacle shapes, we use corresponding signed-distance functions.

¹See [18] for a complete construction of higher-order CBFs.

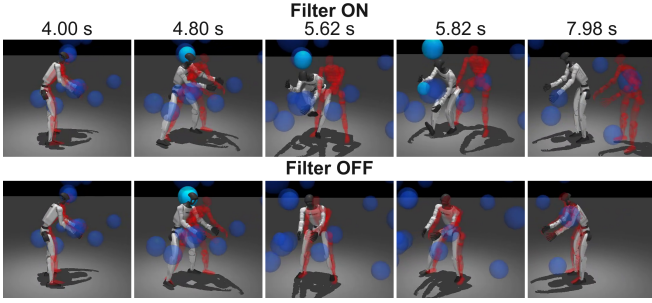


FIG. 2: Tracking with the filter on (top) and off (bottom) with the same base policy and scene. White: robot. Red: reference. Cyan: barrier violation.

The torque input $u = \tau$ enters the condition through the contact-consistent model. Each CBF constraint yields a row $G_{i,t} = -L_g h_{2,i}$ with $b_{i,t} = L_f h_{2,i} + \alpha_2(h_{2,i})$. Stacking these constraints together with actuator bounds gives the quadratic program:

$$\begin{aligned} & \underset{\tau, \delta}{\text{minimize}} && \frac{1}{2} \Delta \tau^T W_t \Delta \tau + \lambda_{QP}^T \delta \\ & \text{subject to} && G_t \tau \leq b_t + \delta, \\ & && \Delta \tau = \tau - \tau_t^{\text{nom}} \\ & && \delta \geq 0. \end{aligned} \quad (3)$$

Here, τ_t^{nom} denotes the bounded nominal torque proposed by the tracker, $\lambda_{QP} > 0$ penalizes constraint relaxation, and $W_t \succeq 0$ is the task metric that weighs corrective effects on contact force and task/posture accelerations. The solution (τ_t^F, δ_t^*) provides the filtered torque and slack variables. The filtered torque is then mapped back to joint targets.

Given this fixed runtime filter, our objective is to fine-tune the pretrained tracker so that its nominal commands lead to better constraint adherence and require smaller corrective interventions, while limiting losses in tracking accuracy and motion quality (smooth, coordinated motion without disruptive oscillations or chattering). Because zero-order hold, model uncertainty, and QP relaxation can still produce violations during execution, we evaluate realized constraint satisfaction empirically rather than claim an unconditional safety guarantee. To respect the planner-tracker abstraction, we focus on handling local interference while maintaining coordinated motion, rather than expecting even a fine-tuned tracker to replan around fully blocking obstacles.

IV. FAILURE MODES OF INDEPENDENTLY DESIGNED POLICIES AND SAFETY FILTERS

Base trackers were trained with a nominal unconstrained model of the actuators, but applying filters at runtime introduces a distribution shift that impacts the tracking and stability performance of the underlying policy. We distinguish three interacting mismatches in the composed policy-filter loop, regarding dynamics, objective, and information.

A. Dynamics Mismatch

CBFs produce endogenous and autocorrelated perturbations on unfiltered dynamics. Prior work has shown that CBF-QPs can prevent convergence to a safe target by introducing stable equilibria elsewhere [19], or maintain barrier satisfaction while state components not directly constrained by the

barrier diverge [20]. For whole-body humanoid tracking, the corresponding concerns are whether enforcing clearance preserves both progress toward the reference and whole-body stabilization.

This can be particularly problematic. Dynamic balance requires coordinated posture and ground support beyond what a torso-clearance barrier specifies. The task-space QP addresses this by distributing corrections across joints while limiting changes to contact forces, motion, and posture (Section III-B). But minimizing this instantaneous intervention does not guarantee that the pretrained tracker will maintain coordination in its subsequent response. Fig. 2 (4.80–5.62 s) shows an illustration of why that response matters. The torso-CBF interaction is followed by a sharp forward bend and raised knee, unlike the unfiltered comparison. The raised knee also highlights how a disturbed response can affect the filter itself. Mismatched dynamics can cause frequent and unexpected contact transitions, leading to increased uncertainty over contact mode (and thus which support dynamics model to use), which can further disturb balance [7].

With multiple distinct constraints, these issues compound. In Fig. 2 (5.82 s), the hand encounters another constraint while this whole-body response is still occurring. The motion used to regain coordination is itself subject to correction, so the tracker must recover under continuing intervention. By Fig. 2 (7.98 s), the robot is more upright but remains displaced from its reference. This instability clearly motivates fine-tuning with the filter active, to help the tracker learn coordinated responses to interventions [10]. More subtly, this supports pairing correction and motion histories to help the tracker account for the compounding effects of filter interventions.

B. Objective Mismatch

A tracker’s reference pursuit can often be incompatible with safety, and while the filter can suppress instantaneously unsafe actions, it cannot guide the policy’s objective towards safety. For example, in the forward reach of Fig. 1 (left), the reference enters the obstacle. Keeping the hand outside yields a tracking error that encourages the policy to reach forward again. Prior work identifies such persistent controller-filter opposition as a source of chattering and oscillations [10], which we observe in our example. The baseline continues forward with violent oscillations and displaces the ball. And, even in cases without constraint violations (not pictured), we observe pronounced jerky motion, illustrating how persistent controller-filter opposition can compound the whole-body disturbances discussed in Section IV-A. The task is therefore not simply to prevent violation, but to accommodate the blocked reach without repeatedly disrupting the coordination needed for the rest of the motion.

Training on filtered transitions alone does not explicitly reward the policy for reducing intervention. Under filter constraints, different proposals can yield the same executed command and execution-based reward [21]. For a filter-blocked trajectory, this means that proposing motion further into the restriction can be indistinguishable in that reward from proposing less, even though more correction is required.

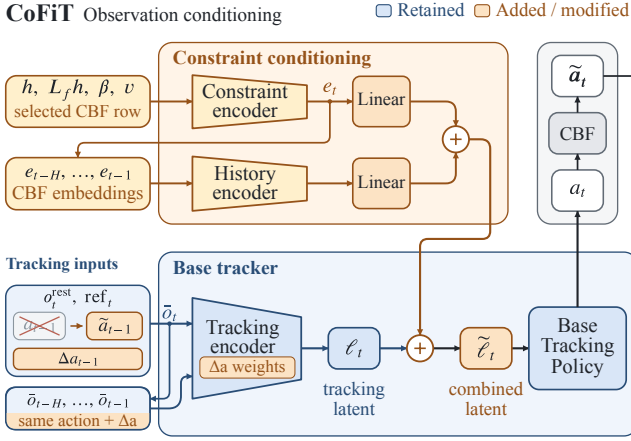


FIG. 3: CoFiT observation conditioning

In the relaxed QP (3), stronger incompatibility can instead increase constraint relaxation without increasing correction, and so the correction cost alone does not penalize increased slack in the solution. These distinctions motivate adding penalties to incompatible proposals and costly corrections, following CBF-RL [9], and additionally penalizing solved slack so the learner is discouraged from leaving barrier conditions relaxed, not merely from demanding large corrections. This also motivates using the *filtered* previous action in policy feedback and in any policy-native action-rate penalty.

C. Information Mismatch

Simply rewarding compatibility does not tell the tracker which actions the current constraints permit. Consider a hypothetical variation of Fig. 1: hold the robot state, reference, and complete native input history fixed, but place the sphere just left or right of the upcoming hand path. Before either placement elicits correction, the tracker produces the same action distribution, although a lateral adjustment that increases clearance in one scene decreases it in the other. A penalty can favor avoiding intervention in both scenes, but identical inputs give the policy no basis for distinguishing the placements. Past corrections cannot supply that distinction before either scene has changed a command. Thus, the policies should be aware of constraint information, even before the constraints become active. Further, the provided information should allow the policy to discern which action directions will lead to violation, as well as constraint state and approach rate (for our example, how far the hand is from the ball and how quickly it approaches). Further, to help alleviate the dynamics mismatches discussed in Sec. IV-A, we should supply the policy a history of this constraint information.

V. FILTER-AWARE POLICY DESIGN

CoFiT (illustrated in Fig. 3) fine-tunes whole-body trackers to cooperate with a safety filter while maintaining performance. We retain the base tracker’s MLP structure and native tracking rewards as a backbone, with the ConstrainedMimic whole-body filter active throughout training and deployment [7].

A. Constraint-Context Observations

As motivated in Sec. IV-C, we endow the base policy with information regarding which directions would lead to violation, and how urgent the constraint is.

To give the tracker this relationship directly, we express the inequality constraint in policy-action coordinates using the torque map $\tau = B_t a + c_t$ before target and torque clipping. Here, $a \in \mathbb{R}^m$ is the policy action, and B_t, c_t are the gain and offset of the proportional-derivative controller at the current state z_t . Substituting this map into the CBF rows of (3) gives:

$$\begin{aligned} v_{i,t}^\top a + \beta_{i,t} &\geq 0, \\ v_{i,t}^\top &= -G_{i,t} B_t, \\ \beta_{i,t} &= b_{i,t} - G_{i,t} c_t. \end{aligned} \quad (4)$$

The constraint normal $v_{i,t}$ points toward action changes that push the system away from the constraint while $\beta_{i,t}$ is the constraint margin at zero action.

We also include the barrier value h_i and rate $L_f h_i$, which describe the constraint’s current state and approach.

The resulting observation is

$$\chi_{i,t} = [h_i; L_f h_i; \hat{\beta}_{i,t}; \hat{v}_{i,t}] \in \mathbb{R}^{m+3}, \quad (5)$$

where hats denote divisions by $\|v_{i,t}\|_2$ to normalize inputs.

This requires no additional expensive differentiations, since the QP already computes this information. CoFiT selects the constraint with the smallest barrier value and encodes it into e_t with a small MLP (the filter continues to enforce all constraints). Because this observation does not rely on specific constraint geometry, it shares meaning across *any relative degree two CBF*.

B. Temporal Feedback and Policy Integration

Constraint history describes how local restrictions evolve along the motion, while correction history records how the filter changed the tracker’s proposals. Let q_t^{prop} denote the target after action scaling, offset, and target clipping, but before correction. Comparing it with the returned q_t^{cmd} gives the feedback correction and action:

$$\Delta a_t = \frac{q_t^{\text{cmd}} - q_t^{\text{prop}}}{s_a}, \quad \tilde{a}_t = a_t + \Delta a_t, \quad (6)$$

where division by the action scale s_a is elementwise. We use $\tilde{a}_{t-1}$ in the native previous-action channel and as the reference for the action-rate term $a_t - \tilde{a}_{t-1}$, so both refer to command feedback after filtering.

We append the latest correction Δa_{t-1} to the native observations to form $\bar{o}_t$, and retain the $H = 10$ preceding frames of both $\bar{o}_t$ and the encoded context e_t . Both histories reset at episode boundaries, and stored context embeddings retain their collection-time values without gradient flow.

The current context and its encoded history are added to the first hidden layer of the base policy (See Fig. 3). We initialize all new weights to zero so that before any fine-tuning, the augmented network behaves identically to the base policy. All of these changes apply to both the actor and the critic (each learning a separate network).

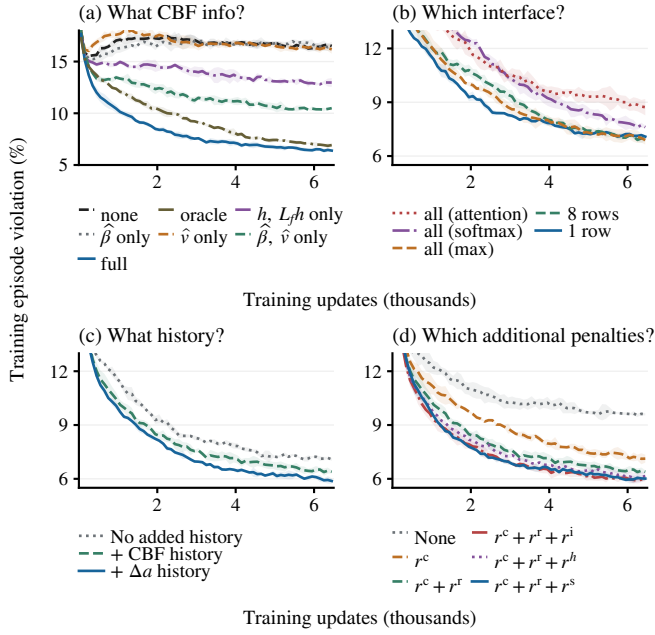


FIG. 4: Curves average three seeds after 100-update windows, with bands showing sample SD.

C. Policy-Filter Alignment Rewards

We add three filter-related penalties to the base policy. The first two are adaptations of CBF-RL’s correction and raw-condition rewards, modified for a whole-body task-space filter metric [9]. Thirdly, because the QP in (3) may relax constraints when the problem would otherwise be infeasible, we penalize the largest solved slack.

Letting $\Delta\tau_t = \tau_t^F - \tau_t^{\text{nom}}$ be the torque-space filter intervention and using the QP rows from (3), the added penalties are:

$$\begin{aligned} r_t^c &= \exp\left(\frac{-\Delta\tau_t^T W_t \Delta\tau_t}{\sigma^2}\right) - 1, \\ r_t^r &= \min\left\{0, \min_{i \in \mathcal{V}_t} (b_{i,t} - G_{i,t} \tau_t^{\text{nom}})\right\}, \\ r_t^s &= -\max_{i \in \mathcal{V}_t} (\delta_{i,t}^*). \end{aligned} \quad (7)$$

Here, $\sigma > 0$ sets the correction scale, $\mathcal{V}_t$ contains CBF constraint rows, and $\delta_{i,t}^*$ is the solved slack.

Retaining native reward terms and adding these penalties, our reward becomes:

$$\bar{r}_t = r_t^{\text{native}} + \lambda_c r_t^c + \lambda_r r_t^r + \lambda_s r_t^s. \quad (8)$$

VI. EXPERIMENTS

We evaluate CoFiT through four outcomes: realized safety, cooperation with the filter, motion quality, and reference tracking. We first isolate design choices in a simplified setting, then validate on more realistic scenes, a second tracker, and hardware.

We provide the tracker with fixed world-frame trajectories that cannot replan in response to tracking errors or safety interventions. Our scenes are therefore designed as near-misses or mild interferences requiring local readjustment rather than rerouting. Because drift in lateral position or heading may turn these into planning-level problems, we

TABLE I: Validation on sphere scenes with the runtime filter. Means with sample SD in subscripts. Lower is better.

	Condition	Violation time (%)	Correction RMS (rad)	World MPJPE Filtered (cm)
Baselines	Pretrained	4.13 _{n=1}	0.321 _{n=1}	5.47 _{n=1}
	Unfiltered training	5.25 _{0.51}	0.372 _{0.016}	5.30 _{0.16}
	Filter-only	8.00 _{0.24}	0.540 _{0.012}	6.02 _{0.03}
Interface	1 constraint	1.22 _{0.03}	0.191 _{0.010}	5.88 _{0.04}
	8 constraints	1.63 _{0.07}	0.185 _{0.010}	5.95 _{0.07}
	Pooled	1.55 _{0.23}	0.178 _{0.010}	6.07 _{0.03}
Penalties Added to $r^c + r^r$	+ intermediate	0.88 _{0.05}	0.201 _{0.004}	6.36 _{0.10}
	+ violation	0.97 _{0.06}	0.204 _{0.000}	6.37 _{0.08}
	+ slack	0.94 _{0.04}	0.197 _{0.004}	6.37 _{0.12}
CoFiT		0.58_{0.02}	0.173_{0.010}	6.25_{0.06}

Three training seeds except pretrained ($n=1$). Each seed evaluates 4,096 initial episodes through first termination. Violation time is the percent of recorded steps with any violation; correction RMS is the joint-command correction norm.

use a world-position-aware tracker trained from scratch with TWIST2’s recipe. We observed less native drift with SONIC and, given its higher pretraining cost, use its publicly available tracker module to initialize fine-tuning [1], [2].

TWIST2 training uses a custom MJLab environment [22], while SONIC is fine-tuned in its native Isaac Lab [23]. Both use filters constructed in CBFpy [17] and an elastic solver adapted from QPAX [24].

Metrics

We measure safety by violation time, the percentage of recorded samples with any negative physical clearance; filter cooperation by correction RMS, the root-mean-square L_2 norm of joint-target corrections. Motion quality is measured by residual acceleration as the RMS acceleration of joint-position tracking error, computed by second differences, and by command total variation (TV), changes between consecutive applied joint targets. Tracking accuracy is measured by world-frame mean per-joint position error (MPJPE) over the tracked key-body positions.

A. Controlled Design Study on Sphere Scenes

We sample motions from TWIST2’s training corpus and place spherical obstacles along their reference trajectories, close enough that the tracker must accommodate them while continuing the motion (see example in Fig. 2). We compare 29 configurations over three training seeds each to specify the details of CoFiT’s observation, history, and reward design (following the broad insights from Section IV), and report the most informative comparisons. While we care about more than safety, it remains the first priority. Thus, we use the training violation-rate curves in Fig. 4 as our primary selection tool, deferring to other metrics only as needed. Note that training violations are calculated under exploration noise and domain randomization, so their values are higher than and not directly comparable to deterministic validation numbers in Table I.

What constraint information does the policy need? (Fig. 4a)

We ablate subsets of the observation package given in (5), with the encoder, constraint history, and rewards fixed. We find neither half of the affine constraint to be useful alone: supplying only the condition margin $\hat{\beta}$ or only the action-space constraint normal $\hat{v}$ performs no better than supplying no constraint information at all. Supplied together, they substantially reduce training violations. The barrier value and rate also help on their own: they describe the current clearance and approach, but do not specify how candidate actions affect the constraint. The complete package achieves the lowest training violation among the tested subsets. It also reaches lower violation, and does so faster, than the oracle-observation baseline given complete world-frame knowledge of the obstacles. These results support the joint use of constraint-state and action-condition information in this setting.

How should constraint information be integrated? (Fig. 4b)

We compare ways of integrating the constraint observation with rewards fixed and no constraint history. Attention queries all the encoded constraints using the remaining policy input. Softmax and max pooling aggregate all encoded constraints. Top- k selects the k constraints with the smallest barrier values and encodes them directly, zero-padding when fewer valid constraints are available. Attention and softmax yield the highest training violation rates. The remaining three are essentially tied on training safety. We choose top-1 as it is slightly safer in validation (Table I) and simpler.

What historical information is useful? (Fig. 4c)

Both base trackers are provided a short window of proprioceptive history, which lets a policy infer unobserved properties of its own dynamics from how the body responded to recent commands [25], [26]. As discussed in Sec. IV-A, the filter perturbs these dynamics. Holding the nearest-constraint description and rewards fixed, we cumulatively add constraint history, then correction history (the tracker already receives the filtered previous action and a single step of the correction). We find that each addition reduces violation time.

How should the reward be modified? (Fig. 4d)

We ablate the added penalties while retaining the complete nearest-constraint description and its history. Penalizing the task-space correction the filter applies (r^c) captures most of the available gain. Adding a penalty on proposals that violate the condition before correction (r^r) helps further (nontrivially). Beyond those two, the benefits are incremental: three further penalties, on the intermediate condition (r^i), the realized violation (r^h), and the largest slack the solver allowed (r^s), land close to one another and close to the pair (Table I), with r^i marginally safest (each new reward was weighted to contribute similarly). What matters is that the policy receive *some* signal about what the filter had to do. We choose to use r^s , since it alone measures the solver’s behavior rather than the robot’s state. A practitioner may reasonably choose otherwise.

The CoFiT recipe

Thus, these design ablations shape our chosen recipe for CoFiT: the nearest constraint’s complete description, histories of that description and of the filter’s corrections, and

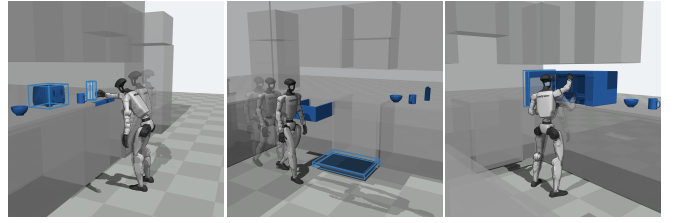


FIG. 5: Generated constraint scenes with overlaid reference poses and task-relevant obstacles highlighted in blue.

the correction, raw-condition, and slack penalties. Against continued unfiltered training it reduces violation time by 89% and the correction the filter must apply by more than half, for 0.9 cm more world-frame tracking error (Table I).

B. Constrained Tracking Across Scenes and Policies

We next evaluate the CoFiT design on generated constraint scenes by fine-tuning TWIST2 and SONIC separately.

We use a mix of manipulation, locomotion, and locomotion reference motions generated via Kimodo [27], conditioned on language descriptions of household tasks. To generate scene geometry, we sweep the robot’s collision bodies along each reference and place obstacles relative to the resulting volumes, so every scene is built around the motion it constrains. Obstacles are spheres, boxes, and capsules, each paired with the bodies it restricts, drawn from free-space layouts and kitchen geometry from RoboCasa [28] (see Fig. 5). Unlike the sphere scenes, we let obstacles overlap the reference itself, so the tracker must depart from its prescribed reference to remain safe. We fine-tune each backbone separately over three seeds, comparing CoFiT with observations-only (our observation package with native rewards), penalties-only (our added penalties without constraint observations), filter-only fine-tuning, and continued training without filtering.²

We find that CoFiT improves constraint adherence and policy-filter cooperation in both trackers. With the filter active for all five conditions, CoFiT achieves the lowest mean violation time and correction RMS in both trackers. Relative to filter-only fine-tuning, violation time falls by 91% for TWIST2 and 21% for SONIC, and correction RMS by 59% and 29%, respectively (Table II). The five treatments also have the same ordering on both measures in the two trackers (CoFiT, observations-only, penalties-only, filtered training, and unfiltered training). Further, CoFiT provides smooth motion, giving the lowest mean command variation in both trackers, together with lower residual acceleration than filter-only training.

The observation-only comparison explains much of this improvement. In both trackers, observations-only outperforms penalties-only on violation and correction. More importantly, adding our observations to filter-only training improves constraint adherence without higher mean world-frame tracking error, while also reducing residual acceleration and command variation, with the native rewards unchanged. The

²For all remaining TWIST2 results, CoFiT uses $\lambda_c = 1$, $\lambda_r = 0.005$, $\lambda_s = 0.015$, and $\sigma = 0.5$. For SONIC we scale the penalty weights by the ratio of average returns between the two trackers.

TABLE II: Constrained tracking in generated scenes. Mean with sample SD in subscripts across training seeds.

Backbone	Method	Viol. (%)	Max depth (p95 cm)	Correction (rad RMS)	World MPJPE (cm)	TV (rad/step)	Resid. acc. (rad/s ² RMS)
TWIST2	CoFiT	1.71_{0.10}	8.27_{0.55}	0.188_{0.003}	4.33 _{0.06}	0.174_{0.003}	18.8 _{0.3}
	Observations-only	5.01 _{0.22}	13.50 _{0.27}	0.251 _{0.007}	3.54_{0.04}	0.197 _{0.007}	18.3_{0.5}
	Penalties-only	15.48 _{0.94}	11.34 _{0.64}	0.343 _{0.013}	5.45 _{0.24}	0.180 _{0.014}	19.3 _{0.4}
	Filter-only	19.31 _{0.58}	14.77 _{0.28}	0.458 _{0.016}	3.77 _{0.08}	0.259 _{0.014}	19.6 _{0.1}
	Unfiltered training	23.75 _{0.27}	15.58 _{1.05}	0.857 _{0.023}	4.08 _{0.03}	0.212 _{0.007}	24.4 _{0.8}
SONIC	CoFiT	11.27_{0.35}	5.31 _{0.40}	0.369_{0.021}	13.07 _{0.28}	0.158_{0.004}	15.5_{0.5}
	Observations-only	12.40 _{0.52}	4.95_{0.11}	0.405 _{0.013}	12.53 _{1.30}	0.162 _{0.001}	15.6 _{0.1}
	Penalties-only	13.38 _{0.38}	5.20 _{0.19}	0.495 _{0.003}	13.57 _{0.38}	0.196 _{0.005}	16.9 _{0.3}
	Filter-only	14.20 _{0.90}	5.69 _{0.81}	0.517 _{0.016}	12.63 _{0.66}	0.200 _{0.002}	16.3 _{0.7}
	Unfiltered training	20.45 _{0.28}	9.25 _{0.49}	1.202 _{0.008}	7.11_{0.24}	0.457 _{0.008}	32.9 _{0.6}

Three training seeds per method, 2,048 validation episodes per seed. **Bold/italic** denote lowest/second-lowest means within each backbone. SONIC/TWIST2: 2,500/6,500 updates.

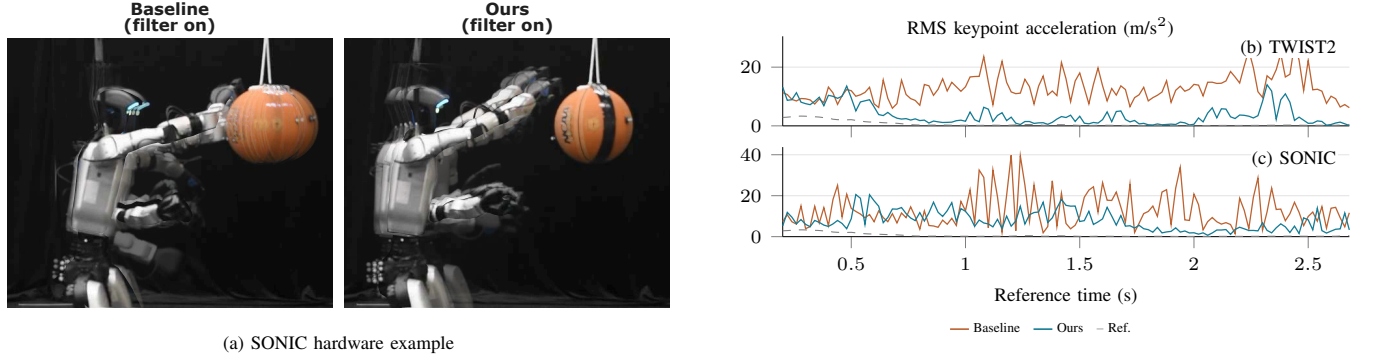


FIG. 6: Hardware examples with the filter active. (a) SONIC overlay comparison. The TWIST2 overlay appears in Fig. 1. (b,c) RMS keypoint acceleration summarizes acceleration of tracked body points, measured during the pictured trials.

TABLE III: Repeated hardware trials. Mean with sample SD in subscripts.

Backbone	Method	Violation (%)	Correction (rad RMS)	World MPJPE (cm)	Command TV (rad/step)	Residual accel. (rad/s ² RMS)	Arm vel. error* (rad/s RMS)	E-stops required [†]
TWIST2	CoFiT	8.13 _{1.98}	0.449 _{0.055}	7.55 _{1.04}	0.381 _{0.057}	31.9 _{2.3}	2.16 _{0.30}	0/10
	Unfiltered Training	46.85 _{30.64}	1.362 _{0.649}	9.38 _{3.18}	0.894 _{0.445}	65.6 _{27.5}	3.29 _{1.11}	5/10
SONIC	CoFiT	5.24 _{2.76}	0.750 _{0.095}	12.17 _{3.22}	0.551 _{0.083}	18.3 _{3.7}	1.05 _{0.21}	0/10
	Unfiltered Training	7.64 _{3.37}	0.801 _{0.109}	5.45 _{0.92}	0.534 _{0.114}	19.8 _{4.1}	1.34 _{0.34}	0/10

Continuous metrics use $n=9$ for SONIC-CoFiT (one logging failure) and $n=10$ otherwise. E-stop counts use all ten trials per condition. The filter is always active. *Calculated over all joints on the reaching arm. [†]Operators used e-stops when the robot thrashed violently.

observations-only tracker therefore learns to demand less correction without an explicit penalty for doing so.

Compared with observations-only training, adding our penalties further reduces violation time, correction magnitude, and command variation in both trackers, at the cost of tracking accuracy. For SONIC, much of the tracking difference from continued unfiltered training is already present after filter-only fine-tuning: world-frame MPJPE is 7.11 cm after continued unfiltered training, 12.63 cm after filter-only fine-tuning, and 13.07 cm with CoFiT.³

Across these varied scenes and both trackers, we replicate the main design lesson of the sphere study: constraint observations improve adherence under native rewards, while added penalties further reduce violation time and corrective effort at a tracking cost relative to observations-only training.

C. Real-Robot Validation

Finally, we test whether the learned policy-filter interaction improves execution on a physical Unitree G1 during a

prescribed reaching task near a basketball. For each backbone, we evaluate CoFiT and the unfiltered-training baseline—both trained on the generated scenes as in Sec. VI-B—as they execute the same motion with the whole-body filter active at 50 Hz. The reference enters the modeled exclusion sphere, requiring the tracker to accommodate the blocked reach while maintaining the rest of the whole-body motion. We include a conservative safety buffer of approximately 4 cm to allow for sim-to-real mismatch, giving a 32 cm-diameter virtual obstacle. *This buffer was fixed before the experiments.* For both trackers, the filter receives the pelvis and basketball poses from motion capture (MoCap), together with robot state for forward kinematics.

As before, our TWIST2-based policy receives the additional global-position feedback, while SONIC does not. Table III reports ten physical repetitions per condition. One SONIC-CoFiT trial had unusable logged data but completed without an operator stop. It is excluded from continuous-metric statistics but included in the stop count. With our TWIST2-based tracker on the G1, we demonstrate improved constraint adherence and tracking together with

³We hypothesize, but do not validate experimentally, that SONIC’s lack of world-frame feedback contributes to this loss.

markedly lower residual acceleration and command variation. CoFiT reduces violation time by 83% relative to continued training, while lowering world-frame tracking error and the correction required from the filter (Table III). Across repeated trials, residual acceleration falls from 65.6 to 31.9 rad/s², while command variation also falls by more than half. The RMS keypoint-acceleration trace in Fig. 6(b) shows much smoother motion for CoFiT than the baseline through the illustrated reach, and the pose overlays in Fig. 1 show the resulting difference in whole-body motion. Five of ten baseline TWIST2 trials required an e-stop due to violent thrashing whereas CoFiT completed all ten without a stop.

Using SONIC on the same G1, we also find reductions in correction RMS, residual acceleration, and reaching-arm velocity error with CoFiT. Mean violation time falls from 7.64% to 5.24%. These gains accompany an increase in world-frame MPJPE from 5.45 to 12.17 cm and slightly higher command variation.⁴

Both policies remain relatively steady in Fig. 6(a), although the baseline makes light contact with the ball. The corresponding acceleration traces in Fig. 6(c) show an improvement in smoothness with CoFiT, though less pronounced than for TWIST2. All ten CoFiT trials complete without an operator stop, as do all ten continued-training trials.

Together, these hardware results show that the gains in constraint adherence and filter cooperation extend to physical execution on both trackers, with every CoFiT trial completing without an operator stop.

VII. CONCLUSION

CoFiT fine-tunes humanoid trackers to improve constraint adherence and cooperation with runtime filters. We find that a compact constraint description already computed by the filter improves cooperation even under native rewards, with correction feedback and penalties providing further gains. The recipe applies across both tested trackers and several obstacle geometries. Future work will integrate CoFiT with safety-aware planners to help humanoids carry out more complex tasks safely.

ACKNOWLEDGMENTS

This work is supported in part by Generative Bionics, and by the National Artificial Intelligence Research Resource (NAIRR) Pilot and the Anvil supercomputer supported by the National Science Foundation (award NSF-OAC 2005632) at Purdue University. Pranit Mohnot was supported by an NSF Graduate Research Fellowship. Thanks to Daniel Morton for insightful discussions throughout the project.

REFERENCES

- [1] Y. Ze, S. Zhao, W. Wang, A. Kanazawa, R. Duan, P. Abbeel, G. Shi, J. Wu, and C. K. Liu, "TWIST2: Scalable, Portable, and Holistic Humanoid Data Collection System," arXiv:2511.02832, 2025.
- [2] Z. Luo, Y. Yuan, T. Wang, C. Li, F. Castañeda, S. Chen, Z.-A. Cao, J. Li, D. Minor, Q. Ben, J. Park, D. Sami, Z. Wang, X. Da, R. Ding, C. Hogg, L. Song, E. Lim, E. Jeong, T. He, H. Xue, W. Xiao, S. Yuen, J. Kautz, Y. Chang, U. Iqbal, L. J. Fan, and Y. Zhu, "SONIC: Supersizing Motion Tracking for Natural Humanoid Whole-Body Control," arXiv:2511.07820, 2026.
- [3] T. He, Z. Luo, X. He, W. Xiao, C. Zhang, W. Zhang, K. Kitani, C. Liu, and G. Shi, "OmniH2O: Universal and Dexterous Human-to-Humanoid Whole-Body Teleoperation and Learning," arXiv:2406.08858, 2024.
- [4] L. Yang, J. Li, and A. D. Ames, "PAC-MAN: Perception-Aware CBF-RL for Whole-Body Safety in Humanoid Dodgeball," arXiv:2607.28623, 2026.
- [5] A. D. Ames, X. Xu, J. W. Grizzle, and P. Tabuada, "Control Barrier Function Based Quadratic Programs for Safety Critical Systems," arXiv:1609.06408, 2017.
- [6] K. P. Wabersich and M. N. Zeilinger, "A predictive safety filter for learning-based control of constrained nonlinear dynamical systems," arXiv:1812.05506, 2021.
- [7] D. Morton, P. Mohnot, and M. Pavone, "Constrained Whole-Body Tracking for Humanoid Robots," arXiv:2606.00374, 2026.
- [8] R. Cheng, G. Orosz, R. M. Murray, and J. W. Burdick, "End-to-End Safe Reinforcement Learning through Barrier Functions for Safety-Critical Continuous Control Tasks," arXiv:1903.08792, 2019.
- [9] L. Yang, B. Werner, M. de Sa, and A. D. Ames, "CBF-RL: Safety Filtering Reinforcement Learning in Training with Control Barrier Functions," arXiv:2510.14959, 2026.
- [10] F. P. Bejarano, L. Brunke, and A. P. Schoellig, "Safety Filtering While Training: Improving the Performance and Sample Efficiency of Reinforcement Learning Agents," arXiv:2410.11671, 2025.
- [11] S. Yang, S. Chen, V. M. Preciado, and R. Mangharam, "Differentiable Safe Controller Design Through Control Barrier Functions," arXiv:2209.10034, 2023.
- [12] M. Wijayatunga, N. Wallace, S. Sukkarieh, and R. Armellin, "Learning Safety-Guaranteed, Non-Greedy Control Barrier Functions Using Reinforcement Learning," arXiv:2602.00366, 2026.
- [13] R. Grandia, A. J. Taylor, A. D. Ames, and M. Hutter, "Multi-Layered Safety for Legged Robots via Control Barrier Functions and Model Predictive Control," arXiv:2011.00032, 2021.
- [14] L. Yang, B. Werner, R. K. Cosner, D. Fridovich-Keil, P. Culbertson, and A. D. Ames, "SHIELD: Safety on Humanoids via CBFs In Expectation on Learned Dynamics," arXiv:2505.11494, 2025.
- [15] C. Khazoom, D. Gonzalez-Diaz, Y. Ding, and S. Kim, "Humanoid Self-Collision Avoidance Using Whole-Body Control with Control Barrier Functions," arXiv:2207.00692, 2022.
- [16] V. C. Paredes and A. Hereid, "Safe Whole-Body Task Space Control for Humanoid Robots," arXiv:2311.08409, 2023.
- [17] D. Morton and M. Pavone, "Safe, Task-Consistent Manipulation with Operational Space Control Barrier Functions," arXiv:2503.06736, 2025.
- [18] W. Xiao and C. Belta, "Control Barrier Functions for Systems with High Relative Degree," arXiv:1903.04706, 2019.
- [19] P. Mestres, Y. Chen, E. Dall'Anese, and J. Cortés, "Control Barrier Function-Based Safety Filters: Characterization of Undesired Equilibria, Unbounded Trajectories, and Limit Cycles," arXiv:2501.09289, 2025.
- [20] J. J. Choi, C. J. Tomlin, S. Sastry, and K. Sreenath, "When Are Safety Filters Safe? On Minimum Phase Conditions of Control Barrier Functions," arXiv:2508.07684, 2025.
- [21] H. Markgraf, S. Sawant, H. Krasowski, L. Schäfer, S. Gros, and M. Althoff, "Safe Reinforcement Learning Using Action Projection: Safeguard the Policy or the Environment?" arXiv:2509.12833, 2026.
- [22] K. Zakka, Q. Liao, B. Yi, L. L. Lay, K. Sreenath, and P. Abbeel, "mjlab: A Lightweight Framework for GPU-Accelerated Robot Learning," arXiv:2601.22074, 2026.
- [23] M. Mittal, P. Roth, J. Tighe *et al.*, "Isaac Lab: A GPU-Accelerated Simulation Framework for Multi-Modal Robot Learning," arXiv:2511.04831, 2025.
- [24] K. Tracy and Z. Manchester, "On the Differentiability of the Primal-Dual Interior-Point Method," arXiv:2406.11749, 2024.
- [25] Z. Zhang, J. Guo, C. Chen, J. Wang, C. Lin, Y. Lian, H. Xue, Z. Wang, M. Liu, J. Lyu, H. Liu, H. Wang, and L. Yi, "Track Any Motions under Any Disturbances (Any2Track)," arXiv:2509.13833, 2025.
- [26] A. Kumar, Z. Fu, D. Pathak, and J. Malik, "RMA: Rapid Motor Adaptation for Legged Robots," arXiv:2107.04034, 2021.
- [27] D. Rempe, M. Petrovich, Y. Yuan, H. Zhang, X. B. Peng, Y. Jiang, T. Wang, U. Iqbal, D. Minor, M. de Ruyter, J. Li, C. Tessler, E. Lim, E. Jeong, S. Wu, E. Hassani, M. Huang, J.-B. Yu, C. Chung, L. Song, O. Dionne, J. Kautz, S. Yuen, and S. Fidler, "Kimodo: Scaling Controllable Human Motion Generation," arXiv:2603.15546, 2026.
- [28] S. Nasiriany, A. Maddukuri, L. Zhang, A. Parikh, A. Lo, A. Joshi, A. Mandlekar, and Y. Zhu, "RoboCasa: Large-Scale Simulation of Everyday Tasks for Generalist Robots," in *Robotics: Science and Systems*, 2024.

⁴As hypothesized in Sec. VI-B, the lack of direct world-frame feedback may contribute to this tracking loss.